KYAMBOGO UNIVERSITY COMPUTING STUDENTS ASSOCIATION (KYUCSA)

Git & GitHub Masterclass

From Fundamental Version Control to Pro-Level Open Source Collaboration

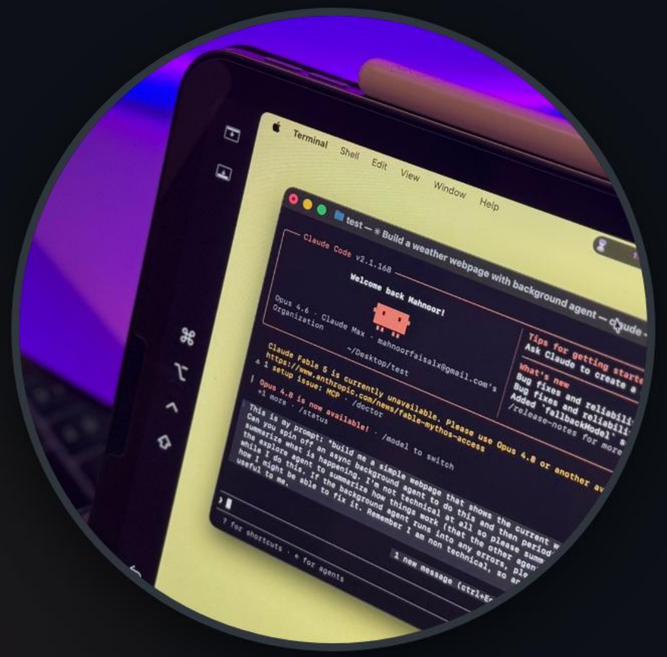
PRESENTED BY

Wambogo Hassan Sadat

MODULE 01

Core Foundations & The Version Control Paradigm

The Version Control Paradigm



Why Traditional Saving Fails

Manual backup strategies like "project_v2_final.zip" quickly collapse in high-velocity software cycles, resulting in overwritten contributions and lost intellectual progress.

Distributed Version Control (Git) preserves a pristine, immutable audit log of your entire source code's historical progression, ensuring zero-loss safety and collaborative seamlessness.

First Steps: Config & Clone

1. Global Config

Identify yourself globally to stamp ownership on commits:

```
git config --global user.name "YourName"
git config --global user.email "your@mail.com"
```

2. Repository Init

Convert an existing local folder into a managed Git workspace:

`git init`

Creates a hidden `.git` repository framework directly inside your directory.

3. Project Cloning

Copy a remote GitHub repository to work locally on your laptop:

`git clone`

Automatically configures remote tracking to easily pull downstream changes.

Safe Connections: HTTPS vs. SSH

- **HTTPS Authentication:** Uses standard web links. Requires credential helpers or GitHub Personal Access Tokens (PATs) for secure command-line interactions.
- **SSH Key Authentication:** Cryptographically secures communication. Requires generating a local public/private keypair (`ssh-keygen`) and registering it to GitHub.
- **The Pro Preference:** SSH handles logins automatically without password prompts, securing CI/CD pipelines and headless environments.

Safeguarding Assets: .gitignore

- **Keep Secrets Secret:** Prevents committing configuration keys, local environments, and API passkeys (e.g., `.env` files).
- **Exclude Node Modules:** Never commit massive generated asset caches like `node_modules/` or system builds.
- **Standard Setup:** Establish wildcards to reject OS cache artifacts such as `.DS_Store` or `thumbs.db`.
- **How it works:** Git checks this file automatically and completely bypasses matched patterns from status trees.

Understanding The Three Git States

1. Working Directory

Your local sandbox where files are created, edited, and dynamically updated before staging.

2. Staging Area

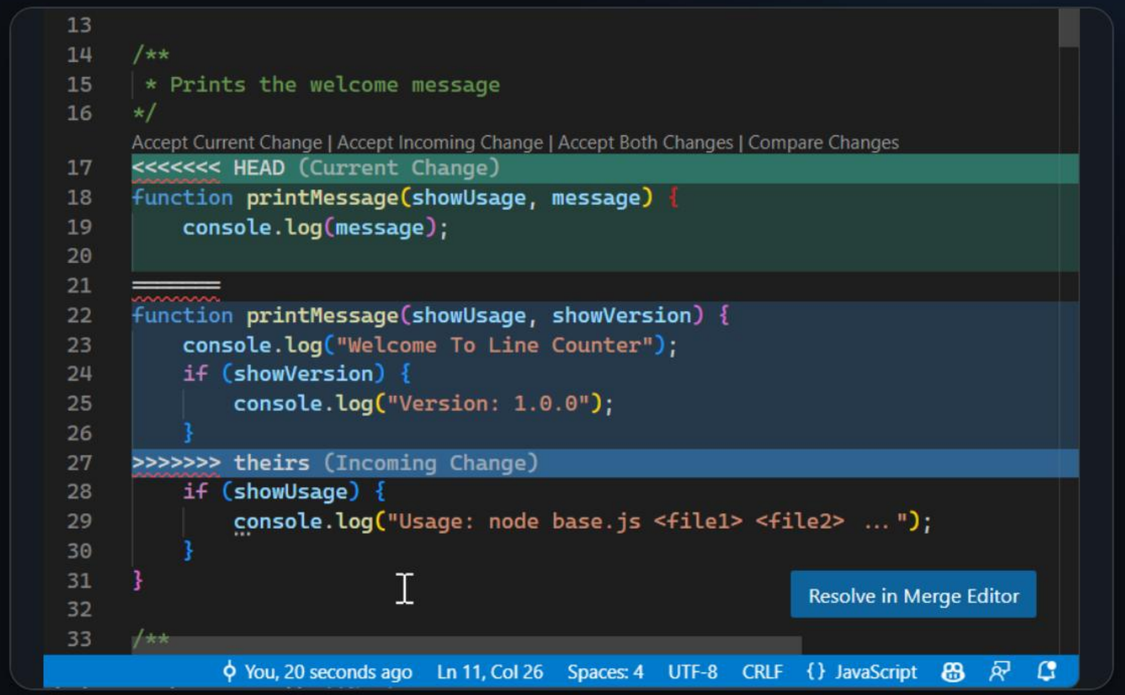
An intermediate index where chosen alterations are indexed, forming the blueprints of your next commit.

3. Local Repository

The permanent historical archive on your disk. Snapshots committed here are secured forever.

The Core Git Command Pipeline

- **git status:** Audits the working tree to show unstaged, staged, and untracked adjustments.
- **git add :** Stages specific files, readying them to form the upcoming snapshot.
- **git commit -m:** Commits the current staged index with a clear documentation message.
- **git push:** Syncs your local branch commits with the corresponding remote GitHub repository.



Syncing Remotes: Fetch vs. Pull

git fetch

Downloads remote commits, refs, and branches from GitHub without modifying your local working files. It update the remote tracking branch (e.g., `origin/main`).

Safety Note: Safe to run anytime. It does not overwrite your current work, allowing you to review remote progress before integration.

git pull

A higher-level command that runs `git fetch` first, then immediately performs a `git merge` to combine the remote tracking branch into your local active branch.

Safety Note: Can trigger auto-merges or unexpected merge conflicts on your files if you have local changes that clash with remote commits.

MODULE 02

Strategic Branching & High-Velocity Teamwork

The Branching Blueprint

Main Branch (Production)

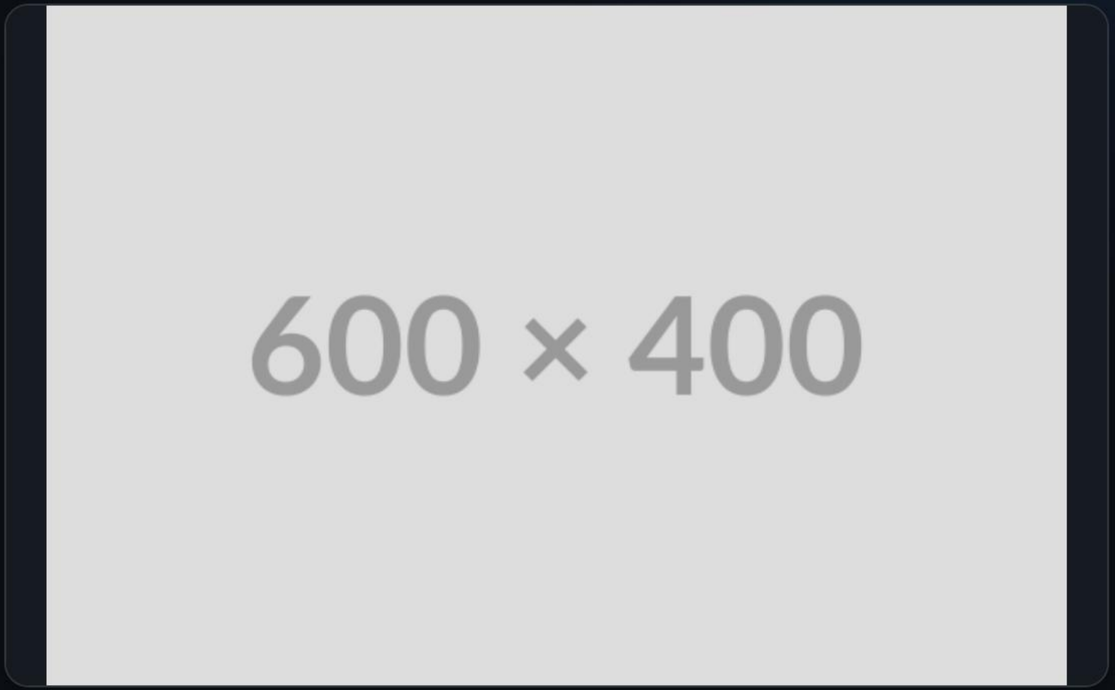
The pristine, high-availability baseline. Direct commits here are strictly prohibited. The main branch is protected to guarantee it is always deployable, fully tested, and secure.

Feature Branches (Development)

Dedicated sandboxes for isolated creation. Developers spawn light, short-lived feature branches to craft specific updates safely without intersecting or breaking colleagues' workflows.

Resolving Merge Conflicts

- **Why Conflicts Occur:** Happens when developers change identical code lines in different branches, and Git cannot automatically merge them.
- **Conflict Markers:** Git highlights contested code blocks inside files using `<<<<<<<`, `=======`, and `>>>>>>>`.
- **Resolution Workflow:** Examine conflicting inputs inside your code editor, select the correct version, save, and stage the solution.
- **Complete Merge:** Seal the resolved fix cleanly on your active terminal by running: `git commit`.

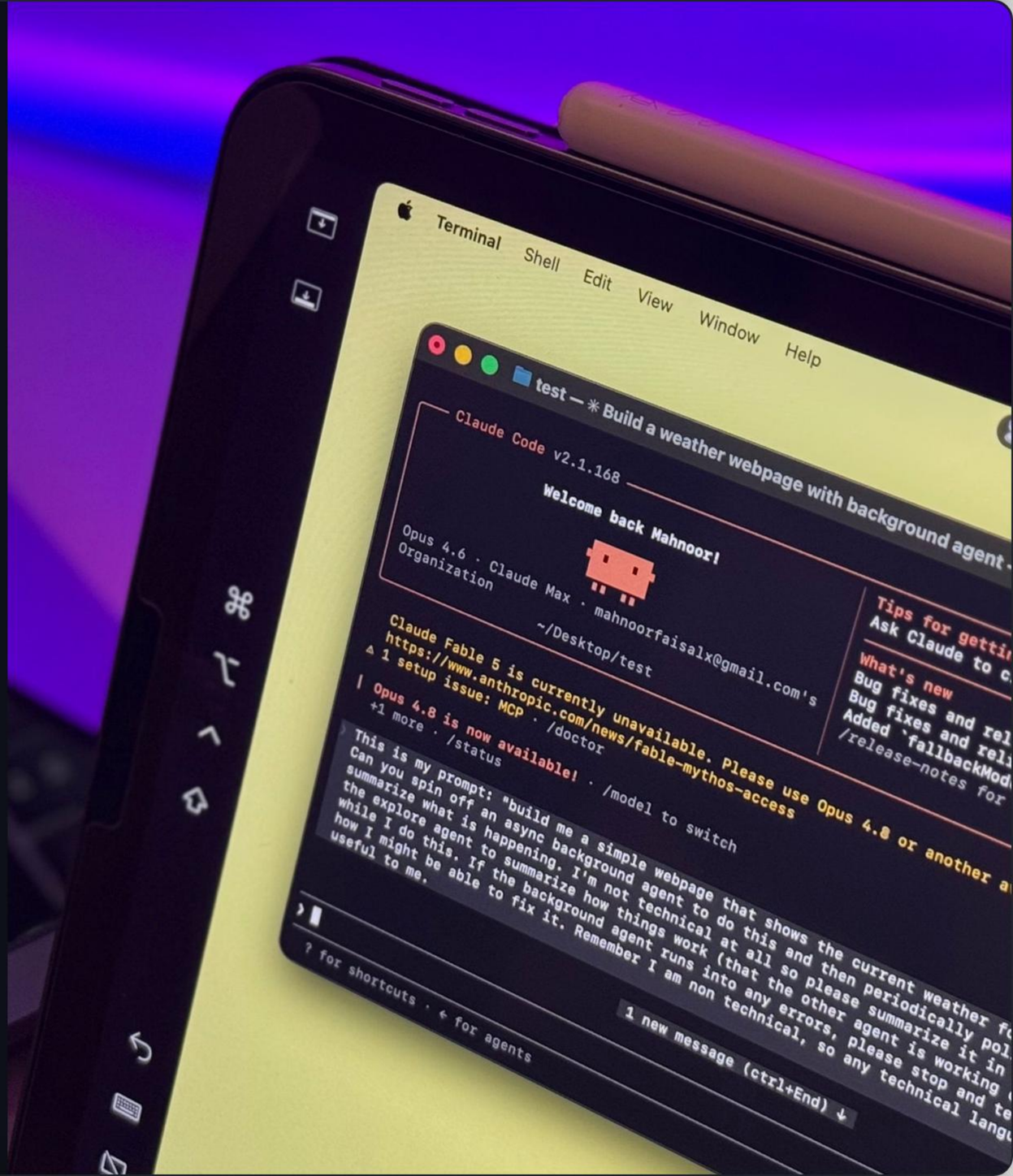


Social Coding: Fork vs. Clone

Collaborative Gateways

Cloning sets up a direct, writable link to your project. **Forking** clones external repositories to your profile, providing a safe sandbox for experimental patches without touching production.

Changes are then proposed back upstream to the original maintainer using a **Pull Request**, introducing standard review, QA diagnostics, and team consensus.

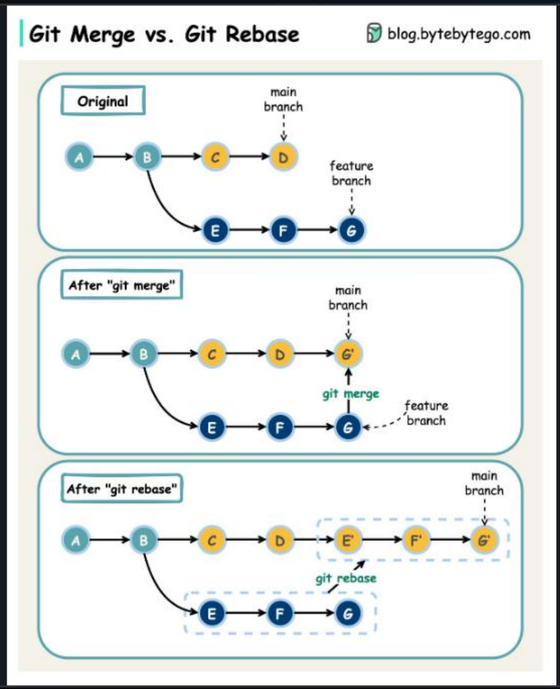


Reset vs. Revert: Handling Mistakes

Mechanism	Target Scope	History Integrity	Best Practice Scenario
git reset	Local Repository	Destructive (Rewrites history)	Cleaning up your private branch before publishing.
git revert	Remote & Shared	Safe (Preserves history)	Undoing errors on shared/production branches.

Branch Integration: Merge vs. Rebase

- **git merge:** Joins development branches together. It preserves full, chronologically accurate team context by adding a dedicated "Merge commit" to the history tree.
- **git rebase:** Rewrites commits on top of another branch base. It produces a perfectly linear, clean commit history by modifying the base ancestor of your commits.
- **The Golden Rule:** Never rebase commits that have already been pushed to public, shared repositories (it disrupts collaborators' trees).



MODULE 03

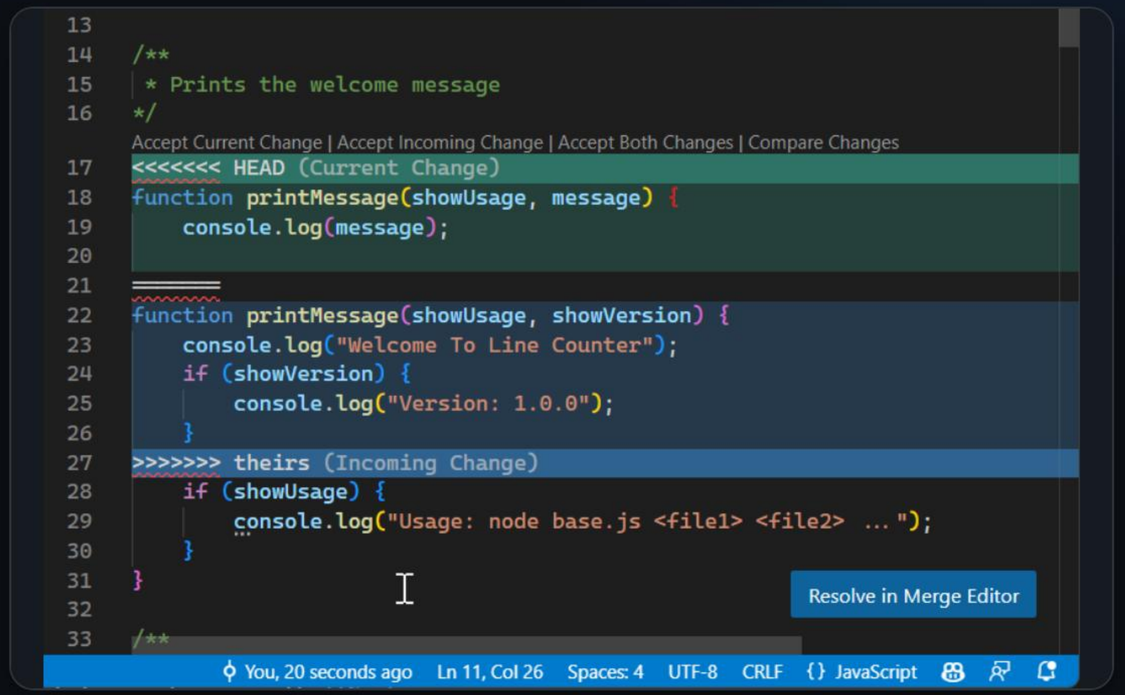
Pro Automation, Stashing & Advanced Integration

Developer Action Breakdown

Up to 30% of engineering resources are consumed by manual testing and deployments. Advanced workflows prioritize automated tools to optimize performance.

Pro Tools: Stash, Log & Recovery

- **git stash:** Pauses work, temporarily shelving modified files so you can switch branches cleanly. Restore via `git stash pop`.
- **git log --oneline:** Audits project logs in a clean, high-density, single-line format for easy tracing.
- **git reflog:** The ultimate safety net. Records every single update made to local repository tips, allowing recovery of deleted commits or branches.
- **git cherry-pick:** Grabs a single commit hash from any development branch and merges it cleanly into active production.



Scaling Deployment via GitHub Actions

Transitioning from manual pipelines to cloud-native GitHub Actions eliminates dev-ops bottlenecks, accelerating deployment iteration speeds.

Questions & Open Discussion

This masterclass is designed to fuel your engineering journey.

wambogohassansadat.dev
wambogohassan63@gmail.com
+256 786 021431
github.com/Chemistry2i

KYUCSA | Kyambogo University Computing Students Association

Image Sources

https://static0.xdaimages.com/wordpress/wp-content/uploads/wm/2026/06/code-editor-window-with-dark-theme-displaying-javascript-or-web-development-code-on-laptop-screen-with-purple-ambient-lighting.jpg

Source: www.xda-developers.com

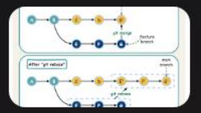
https://i.ytimg.com/vi/MfmWVn5UlLg/hq720.jpg?sqp=-oaymwEhCK4FEIIDSFryq4qpAxMIARUAAAAAGAElAADIQj0AgKJD&rs=AOn4CLA_s2ALH45W0hAbiZrifDYMXWaHYA

Source: www.youtube.com

Thumbnail for code.visualstudio.com

https://code.visualstudio.com/assets/docs/sourcecontrol/overview/merge-conflict.png

Source: code.visualstudio.com



https://assets.bytebytego.com/diagrams/0203-git-merge-git-rebase.jpg

Source: bytebytego.com